

**REAL-TIME SNMP BASED  
DEVICE PROFILING**

**A DISSERTATION**

*Submitted by*

**YESHWANTH K**

**2022239008**

*in partial fulfillment for the award of the degree*

*of*

**MASTER OF SCIENCE (INTEGRATED)**

*in*

**COMPUTER SCIENCE**



**FACULTY OF RAMANUJAN COMPUTING CENTER**

**ANNA UNIVERSITY,  
CHENNAI - 600025**

**DECEMBER 2025**

**ANNA UNIVERSITY,  
CHENNAI 600 025**

**BONA FIDE CERTIFICATE**

Certified that this dissertation titled “**REAL-TIME SNMP BASED DEVICE PROFILING**” is the bona fide work of **Mr. YESHWANTH K** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**(Prof. P. LOGANATHAN)**

PROFESSOR AND HEAD

Department of Mathematics

Anna University

Chennai-600 025

**(Dr. S. LOKESH)**

Associate Professor

Ramanujan Computing Centre

Anna University

Chennai-600 025

## **ABSTRACT**

In modern networked environments, accurately identifying and monitoring devices in real time is essential for security, operational efficiency, and rapid fault resolution. This project focuses on designing a real-time SNMP-based device profiling system that continuously discovers network devices, retrieves their characteristics, and maps them to appropriate monitoring templates. Using SNMP walks, the system extracts key details such as the device's sysOID, hardware information, and performance metrics to create a detailed profile.

The architecture includes automated IP range scanning for discovery, an OID-based profiling engine, a template mapping module, and a real-time dashboard. Unlike traditional setups dependent on static configurations, this system supports both proactive polling and event-driven alerts using SNMP traps to detect performance changes instantly. It supports SNMP v1/v2c and secure SNMP v3, along with modular templates for vendor-independent deployment.

By automatically categorizing devices and adjusting monitoring parameters based on type, the system minimizes manual effort and scales efficiently in large enterprise environments. The solution enhances visibility, speeds fault detection, and enables proactive planning, resulting in more resilient and well-managed network infrastructures.

## திட்டப்பணிச்சுருக்கம்

நவீன நெட்வொர்க் சூழல்களில், சாதனங்களை நேரடியாக அடையாளம் காண்பது மற்றும் கண்காணிப்பது பாதுகாப்பு, செயல்திறன் மற்றும் கோளாறு தீர்க்கும் வேகத்திற்கு மிக முக்கியமானது. இந்த திட்டம், SNMP அடிப்படையிலான நேரடி சாதன புரோஃபைலிங் அமைப்பை வடிவமைத்து செயல்படுத்துவது மீது கவனம் செலுத்துகிறது. இது நெட்வொர்க் சாதனங்களை தொடர்ச்சியாக கண்டறிந்து, அவற்றின் பண்புகள் மற்றும் செயல்திறன் விவரங்களை பெறுவதுடன், சரியான மானிட்டரிங் டெம்ப்ளேட்டுகளுடன் இணைக்கின்றது. SNMP walk-கள் மூலம் சாதனத்தின் sysOID, ஹார்ட்வேர் விவரங்கள் மற்றும் செயல்திறன் அளவுகோல்கள் எடுத்துக்கொள்ளப்படுகின்றன.

கட்டமைப்பில் IP range scanning மூலம் சாதன கண்டறிதல், OID parsing அடிப்படையிலான ஸ்காபிங் engine, template mapping module மற்றும் real-time dashboard ஆகியவை அடங்கும். நிலையான கான்ஃபிகரேஷன்களையே சார்ந்த பாரம்பரிய முறைகளுக்கு மாறாக, இந்த அமைப்பு proactive polling மற்றும் SNMP traps வழியாக event-driven alert-க்களையும் ஆதரிக்கிறது. இது SNMP v1/v2c மற்றும்

பாதுகாப்பான SNMP v3 ஐ ஆதரிக்கிறது, மேலும் vendor-independent deployment க்கான modular templates வழங்கப்படுகிறது.

சாதன வகையை அடிப்படையாகக் கொண்டு தானியங்கி வகைப்படுத்தல் மற்றும் monitoring அளவுருக்களின் தன்னிச்சையான சரிசெய்தல் மூலம், கைமுறையான முயற்சி குறைக்கப்படுகிறது மற்றும் பெரிய enterprise நெட்வொர்க்களிலும் அளவிற்கு ஏற்ப விரிவாக்க முடியும். இந்த தீர்வு காட்சியளிப்பை மேம்படுத்தி, கோளாறு கண்டறியும் வேகத்தை அதிகரித்து, capacity planning-க்கு உதவுகிறது, இதன் மூலம் நெட்வொர்க் இன்னும் வலுவானதும் நன்கு நிர்வகிக்கப்பட்டதுமானதாக மாறுகிறது.

## ACKNOWLEDGEMENT

I would like to express my gratitude to **Prof. P. LOGANATHAN**, Professor and Head, Department of Mathematics, Anna University, Chennai, for his constant motivation and unwavering support throughout the course.

I express my sincere gratitude to my Faculty Advisor and Project Coordinator **V. KANAKASABAPATHI**, Teaching Fellow, Department of Mathematics, Anna University, Chennai, for his continuous motivation and support.

I am also thankful to my Review Faculty Member **K. KAMALAKANNAN**, Teaching Fellow, Department of Mathematics, Anna University, Chennai, for his valuable guidance and constructive feedback throughout this project.

I am profoundly grateful to my Internal Supervisor **Dr. S. LOKESH**, Associate Professor, Ramanujan Computing Centre, Anna University, Chennai, for his inspirational words, meticulous guidance, and invaluable insights during this project.

Finally, I would like to acknowledge my Project Mentor, **Mr. GOPISELVAM D**, Zoho Corporation Pvt. Ltd., Chennai, for their friendly guidance and care throughout the project. I am also grateful to my parents, friends, and colleagues for their support and encouragement, and I remain ever thankful to the Almighty.

**CHENNAI-25**

**DECEMBER 2025**

**(YESHWANTH K)**

# **CHAPTER 1**

## **INTRODUCTION**

This chapter provides an overview of the project and its significance in modern network management. It highlights the importance of real-time device profiling, monitoring, and fault detection in enterprise networks. The chapter also outlines the project objectives and methodology, offering a roadmap for the upcoming sections.

### **1.1 ORGANISATION PROFILE**

Zoho Corporation Pvt. Ltd. is a global software company providing cloud-based business and IT management solutions. Established in 1996, Zoho develops a wide range of SaaS applications including CRM, finance, HR, collaboration tools, and IT monitoring platforms used by millions worldwide. With a strong focus on innovation, security, and user-centric design, the company delivers scalable and reliable solutions that enable digital transformation for businesses across industries.

### **1.2 OBJECTIVE OF THE PROJECT**

The primary objective of this project is to design and implement a real-time SNMP-based device profiling and monitoring system capable of accurately identifying network devices, extracting key attributes, and mapping them to suitable monitoring templates. The project aims to enhance visibility, automation, and security in enterprise networks by supporting continuous device discovery, dynamic profiling, and threshold-based alerting. Additional goals include reducing manual configuration effort, improving fault detection speed, and enabling scalable performance monitoring across large distributed environments.

### **1.3 PROJECT OVERVIEW**

This project addresses the growing need for intelligent and automated network monitoring systems capable of managing complex, multi-vendor environments. Traditional network monitoring relies heavily on static configurations and periodic polling, which limit responsiveness and scalability. The proposed solution introduces a real-time SNMP-based approach that continuously discovers devices, extracts system identifiers (sysOID), and builds detailed profiles including hardware, firmware, and performance metrics.

The system integrates automated IP range scanning, a profiling engine for OID parsing, a template mapping mechanism for standardized monitoring, and SNMP traps for event-driven alerts. It supports SNMP v1/v2c for backward compatibility and SNMP v3 for secure transmission, ensuring reliability and data integrity. Dynamic attention mechanisms, such as rule-based thresholding and automated template assignment, optimize monitoring accuracy and operational efficiency.

The project follows a structured methodology including device discovery, data extraction, profile mapping, alert generation, and real-time dashboard visualization. Expected outcomes include improved network transparency, faster fault response, reduced manual dependency, and scalable performance management—ultimately contributing to more intelligent and resilient network infrastructure.



## **1.4 ORGANIZATION OF THE REPORT**

**Chapter 1** presents an introduction to the organization, project background, objectives, and an overview of the system and its purpose.

**Chapter 2** includes the system analysis, describing the existing system, proposed solution, project requirements, hardware/software specifications, and tools used.

**Chapter 3** explains the system architecture and design, detailing the structural components and functional workflow of the proposed model.

**Chapter 4** describes the implementation phase, the development process, testing methods, challenges faced, and the results obtained.

**Chapter 5** concludes the report by summarizing key outcomes, highlighting limitations, and suggesting future enhancements to improve the system.

## CHAPTER 2

### SYSTEM ANALYSIS

System Analysis involves examining and understanding the existing network monitoring methods and identifying limitations in traditional approaches. The objective is to determine operational gaps and design an improved solution that enables real-time device discovery, profiling, and monitoring with greater accuracy and automation. This analysis breaks down system components and evaluates how they interact to achieve efficient network management.

#### 2.1 EXISTING SYSTEM

In the existing network monitoring environment, SNMP is commonly used for collecting device status and performance metrics using MIBs and OIDs. Most monitoring platforms rely on periodic polling or SNMP traps to detect performance issues or faults. However, device discovery and profiling are typically manual or semi-automated, requiring administrators to configure IP ranges, add devices manually, and assign monitoring templates one by one. Tools like **New Relic** and **Dynatrace** support performance monitoring, and solutions like **FortiNAC** use SNMP Get requests for profiling, but they lack automated, real-time profiling based on sysOID and dynamic template mapping. As a result, the existing system suffers from slow response to changes, heavy manual dependency, limited scalability, and delayed fault detection.

#### 2.2 PROPOSED SYSTEM

The proposed system introduces a fully automated real-time SNMP-based device profiling framework designed to detect devices dynamically, collect SNMP OID data, and map devices to suitable monitoring templates. By using automated IP-range scanning, SNMP walk-based profiling, real-time polling, and event-based trap handling, the system continuously monitors network performance and configuration changes. Support for SNMP v3 ensures secure communication with authentication and encryption. The system provides a live dashboard for visualization, threshold-based alerting, and template-based monitoring to reduce manual effort. This approach increases visibility,

improves fault resolution time, and ensures scalable monitoring of large, distributed enterprise networks.

### 2.3 PROJECT INITIATION

The project was initiated with the vision of designing an intelligent and scalable device profiling system capable of handling real-time discovery and dynamic monitoring across multi-vendor environments. The goal is to reduce monitoring complexity by automatically identifying devices using sysOID, building detailed profiles through SNMP walks, and assigning templates for standard metric collection. The project enables proactive monitoring through SNMP traps and threshold alerts to detect anomalies instantly. This real-time, automated approach ensures improved accuracy, faster troubleshooting, and enhanced operational efficiency in network management.

### 2.4 SYSTEM SPECIFICATIONS

This section describes the hardware and software requirements necessary for developing and deploying the SNMP-based device profiling system.

**Table 2.1 Hardware Specifications**

| Hardware Component                  | Configuration                  |
|-------------------------------------|--------------------------------|
| System Type                         | 64-bit Operating System        |
| Processor                           | AMD Ryzen 5                    |
| Central Processing Unit (CPU) Cores | 8                              |
| Central Processing Unit (CPU) Speed | >= 2.3 GHZ                     |
| Random Access Memory (RAM)          | 8 GB                           |
| Disk Storage                        | 512 GB                         |
| Network Requirements                | LAN access to target IP ranges |

**Table 2.2 Software Specifications**

| <b>Software Component</b>                | <b>Configuration</b>                     |
|------------------------------------------|------------------------------------------|
| Operating System                         | Windows / Linux (Ubuntu / CentOS / RHEL) |
| Integrated Development Environment (IDE) | Visual Studio Code (VS Code)             |
| SNMP Toolkit                             | Net-SNMP / PySNMP / SNMP4J               |
| Programming Language                     | Java                                     |
| Database                                 | MySQL                                    |
| SNMP Trap Reciever                       | SNMPtrapd or custom trap listener        |
| Library                                  | Transformers                             |

## **2.5 TOOLS AND TECHNOLOGIES USED**

The tools, libraries, and technologies used in the project are listed below:

### **2.5.1 Visual Studio Code**

A lightweight and extensible source-code editor that provides debugging, intellisense, integrated terminal, version control support, and extensions for Python, Go, and networking utilities.

### **2.5.2 Java**

Java is used for implementing device discovery logic, SNMP polling, and OID parsing. Libraries such as SNMP4J enable communication with SNMP agents to retrieve system information, performance metrics, and trap messages.

### **2.5.3 SNMP Toolkit (Net-SNMP / PySNMP)**

Net-SNMP or PySNMP provides command-line and programmatic utilities for SNMP operations including SNMP walk, SNMP get, trap listening, and MIB parsing. These tools are core components for building the profiling and monitoring engine.

#### **2.5.4 SNMPtrapd / Custom Trap Receiver**

Used to receive SNMP traps from network devices for event-driven monitoring and immediate fault notification.

## CHAPTER 3

### SYSTEM DESIGN

System design is the process of crafting the structure and framework of a system to meet specific requirements and objectives. It involves translating user needs and system specifications into a detailed plan that outlines how various components will function together to deliver the desired outcomes. This plan serves as a foundation for development, ensuring that the system is effective, scalable, and user-oriented.

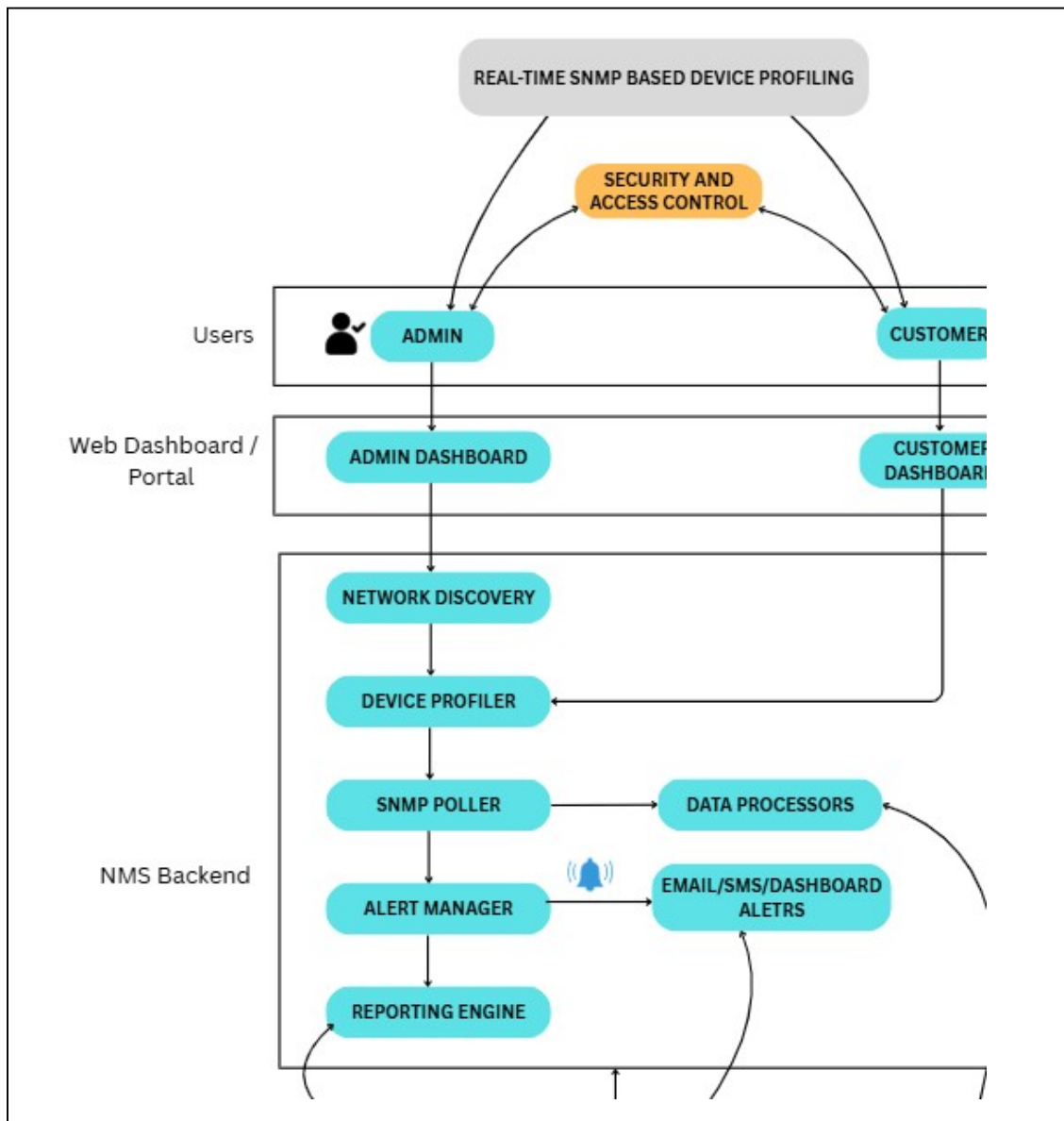
Key aspects of system design include:

- **Logical Design:** Focuses on conceptual frameworks, such as workflows, data relationships, and process diagrams, to define how the system operates at an abstract level.
- **Physical Design:** Deals with tangible elements, including hardware, software, networks, and user interfaces, ensuring the system's practical implementation aligns with design goals.
- **Interface Design:** Defines how system components connect and interact, both with each other and with the users, ensuring seamless communication and usability.

System design plays a pivotal role in creating systems that are robust, efficient, and adaptable. It aligns technical capabilities with user expectations and organizational goals, ensuring the system performs reliably under expected condition.

### 3.1 SYSTEM ARCHITECTURE

System architecture defines the structure and behavior of a system, representing how its components interact and function together to meet specific requirements. It typically includes hardware, software, data, and communication protocols. System architecture is crucial for ensuring scalability, reliability, and efficiency in system design and implementation



**Figure 3.1 System Architecture**

Figure 3.1 illustrates the system workflow, where network devices transmit SNMP data that is stored in the database, including reports, alerts, and performance metrics. The backend module manages device discovery, profiling, and periodic polling, while also generating real-time alerts and comprehensive reports. A secure API layer enables controlled communication between the backend and dashboards. Admin and customer dashboards utilize this API for monitoring, visualization, and management functions with strict authentication and access control mechanisms. Overall, the architecture ensures efficient data flow, security, and scalable network monitoring.

## **3.2 UML DIAGRAMS**

Unified Modeling Language (UML) diagrams are a set of standardized visual representations used to model systems, processes, and their components. They help developers and stakeholders understand system requirements and designs. UML diagrams are categorized into structural diagrams (e.g., Class, component) and behavioral diagrams (e.g., Use Case, Sequence). This project consists of following UML diagrams for virtual representation of the whole system. [7]

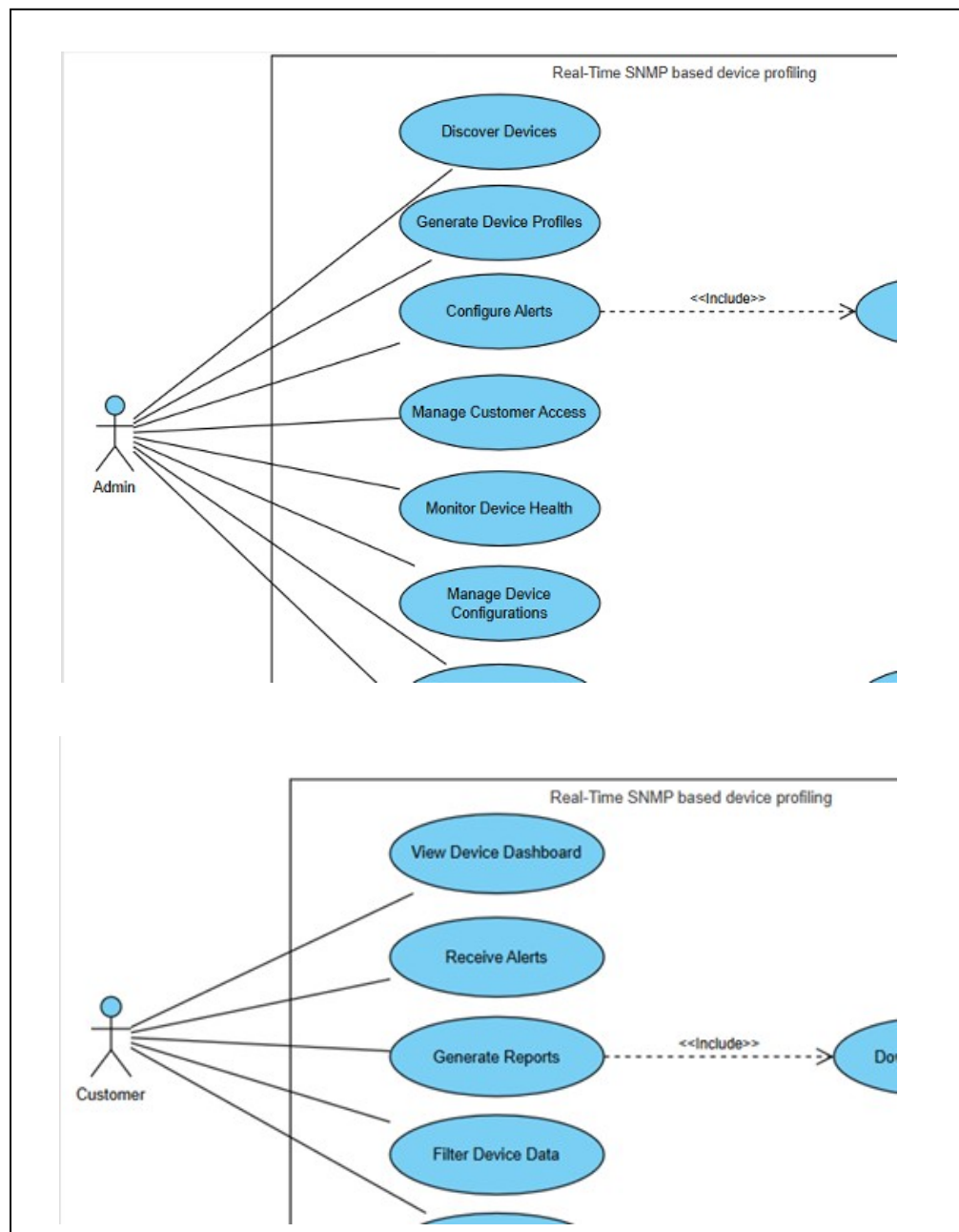
- Use Case Diagram
- Sequence Diagram
- Class Diagram
- ER Diagram

### **3.2.1 Use Case Diagram**

A Use Case diagram represents the interactions between users and the system. It highlights system functionalities and the relationships between actors and use cases.



**Purpose:** To capture system requirements and identify user interactions.



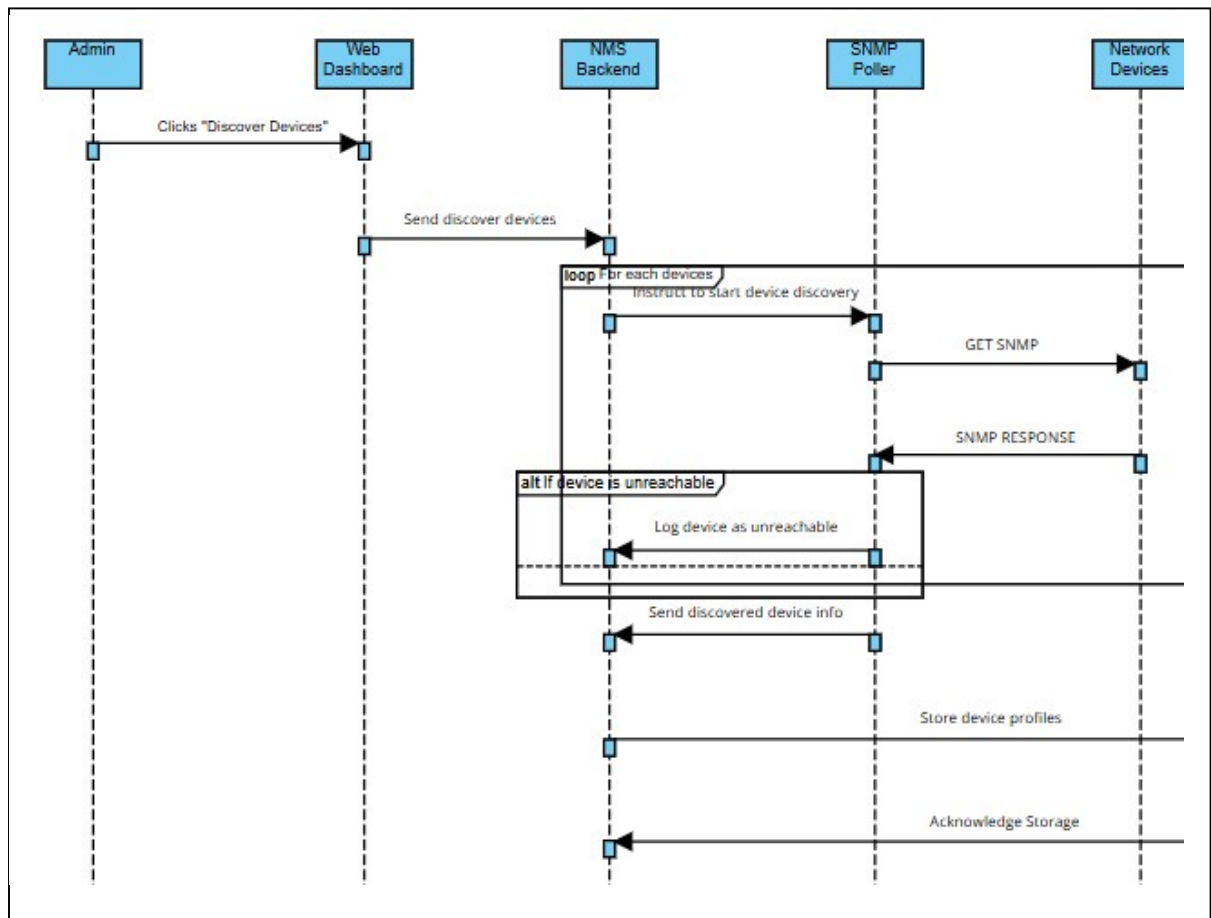
**Figure 3.2 Use case Diagram**

Figure 3.2 represents the Use Case diagram illustrating the interactions between the user and monitoring system module.

### 3.2.2 Sequence Diagram

A sequence diagram illustrates the flow of messages between system components or objects in a specific sequence. It shows how processes interact over time.

**Purpose:** To model dynamic behavior and time-based interactions.



**Figure 3.3 Sequence Diagram**

Figure 3.3 represents sequence diagram illustrating the flow of data and interactions between the user, NMS for discovering the active devices.

### 3.2.3 Class Diagram

A class diagram represents the static structure of a system by modeling its classes, attributes, methods, and relationships.

**Purpose:** To define the blueprint of the system's data and behavior.

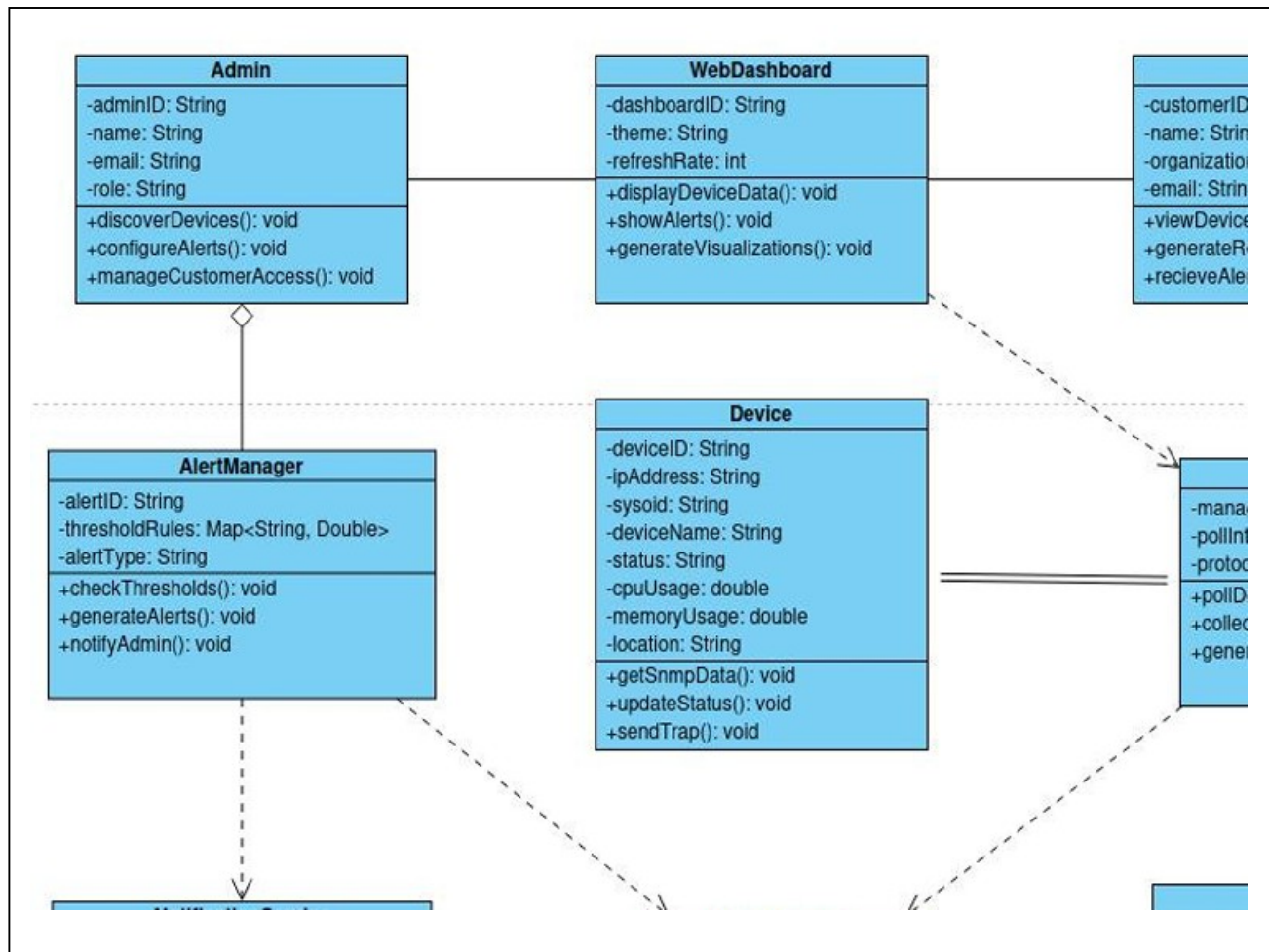


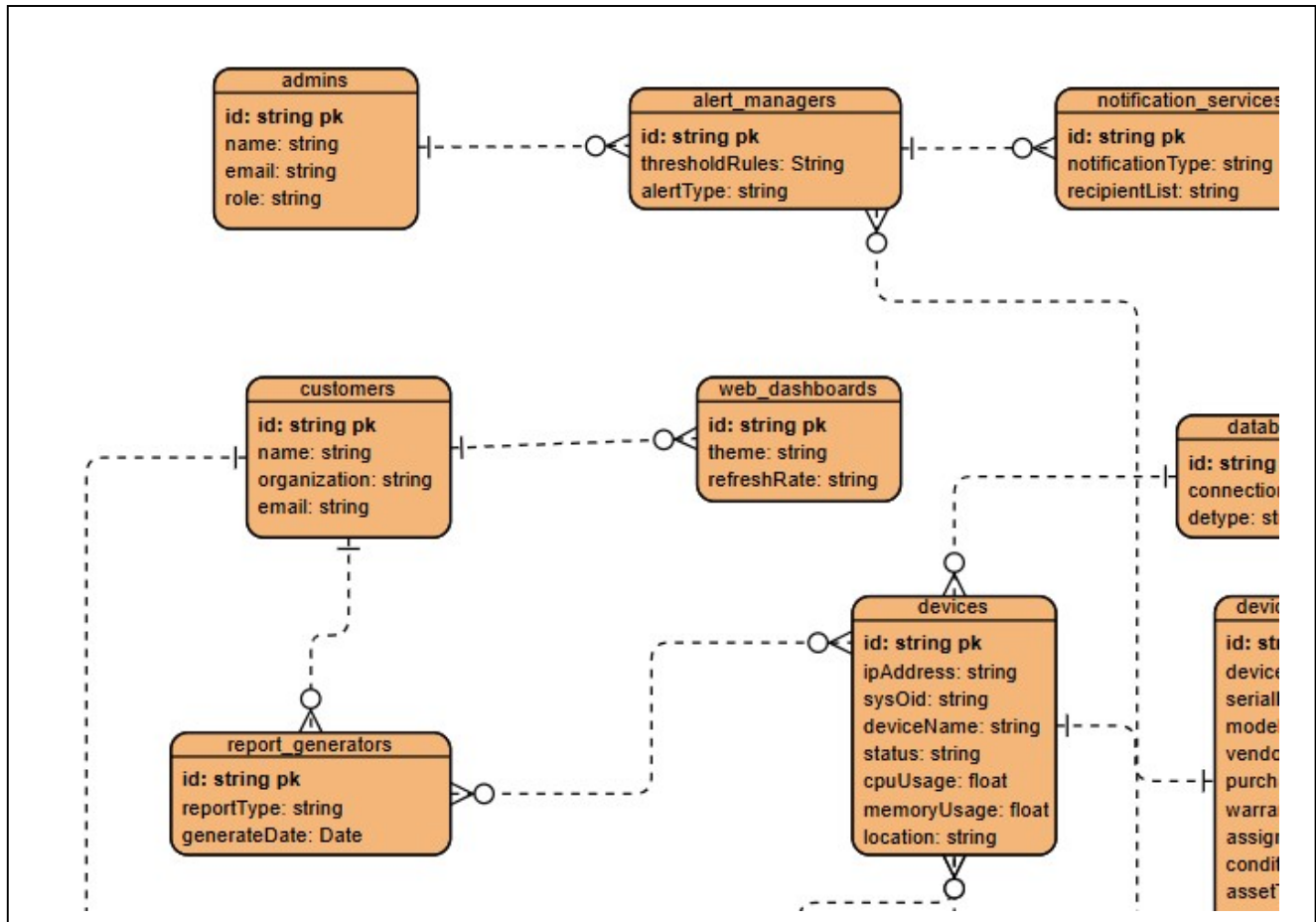
Figure 3.4 Class diagram

Figure 3.4 illustrates the Class diagram showing relationships between the user and Network Monitoring System.

### 3.2.5 ER Diagram

An ER diagram visualizes the database structure by showing entities and the relationships between them.

**Purpose:** To clearly define how data is organized and connected within the system.



**Figure 3.5 ER Diagram**

Figure 3.5 illustrates the relationships between each entities in the Network Monitoring System.

## **CHAPTER 4**

### **SYSTEM IMPLEMENTATION AND TESTING**

The System implementation refers to the process of integrating a designed system into a functioning environment to ensure it meets predefined requirements and operates effectively. It encompasses installing hardware and software, configuring system settings, conducting testing to verify functionality, training users, and transitioning from the old system to the new one. The goal of system implementation is to deploy the system efficiently, address any issues that arise during the process, and ensure that it delivers the intended outcomes in real-world usage.

#### **4.1 SYSTEM MODULES**

A module is a software component or a part of a program that contains one or more routines. One or more independently developed modules make up a program. The project involves the following modules:

- Input/Output Preprocessing Module
- Confidence Estimation Module
- Systematic Response Refinement Module

##### **4.1.1 SNMP Communication Module**

This module enables communication between the system and network devices using SNMP operations. It manages request handling and response processing.

**Functions:**

- Supports SNMP v1/v2c/v3
- Manages retries, timeouts, and error handling
- Executes GET, GETNEXT, and GETBULK requests
- Queries device metadata and performance OIDs
- Optimizes polling to reduce network overhead

**4.1.2 sysObjectID Retrieval and Parsing Module**

Responsible for retrieving and interpreting the sysObjectID, which uniquely identifies device hardware.

**Functions:**

- Fetches OID 1.3.6.1.2.1.1.2.0 (sysObjectID)
- Normalizes vendor-specific OID formats
- Maps sysObjectID to device families
- Sends parsed signature to classification engine

**4.1.3 Device Classification Engine**

Classifies devices based on sysObjectID and MIB data.

**Functions:**

- Matches ID with known vendor/device patterns
- Detects new/unidentified devices dynamically
- Assigns device type (Router, Switch, Printer, etc.)
- Applies heuristic fallback when unknown

#### **4.1.4 OID Discovery & Recommendation Module**

Recommends and maps relevant OIDs for monitoring once device type is known.

##### **Functions:**

- Performs SNMP walk for OID discovery
- Identifies CPU, memory, interface, uptime, temperature metrics
- Maps discovered OIDs to monitoring templates
- Generates recommended profiles for NMS

#### **4.1.5 Metrics Collection & Storage Module**

Collects and stores time-series performance data for trends and reporting.

##### **Functions:**

- Polls metrics at configured intervals
- Validates and normalizes incoming data
- Saves to PERFORMANCE and REPORT databases
- Supplies processed data to dashboards via API

#### **4.1.6 Alert Management Module**

Generates alerts based on critical thresholds and real-time device conditions.

##### **Functions:**

- Evaluates rules (e.g., CPU > 90%, device down)
- Creates alert records with severity details
- Provides email/SMS/dashboard notifications

- Maintains alert history and timestamps

#### **4.1.7 Dashboard & API Integration Module**

Connects backend data to frontend dashboards through secure APIs.

##### **Functions:**

- Exposes REST APIs for metrics, alerts, reports, inventory
- Supports real-time updates through polling/WebSocket
- Provides role-based access for Admin and Customer views
- Ensures responsive UI experience

#### **4.1.8 Standalone & NMS Integration Layer**

Supports plug-and-play deployment, independent use, or integration with existing NMS.

##### **Functions:**

- Integrates smoothly with external systems
- Provides standard interfaces for data exchange
- Supports compatibility with platforms like Site24x7
- Enables scalable reuse



### **4.1.9 Logging & Audit Module**

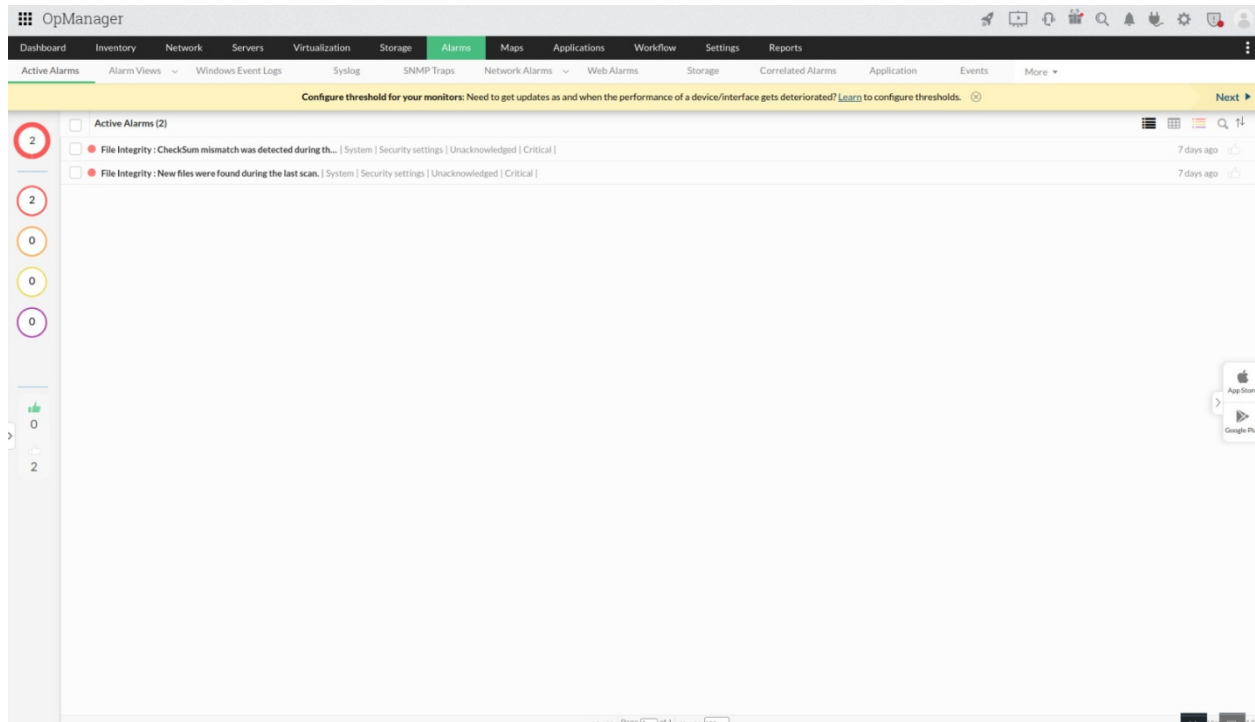
Tracks internal operations for transparency and troubleshooting.

#### **Functions:**

- Logs discovery events, polling cycles, alerts, and admin actions
- Maintains audit trails for compliance
- Supports debugging and performance analysis

## 4.2 SCREENSHOTS

### 4.2.1 Alert Management Module

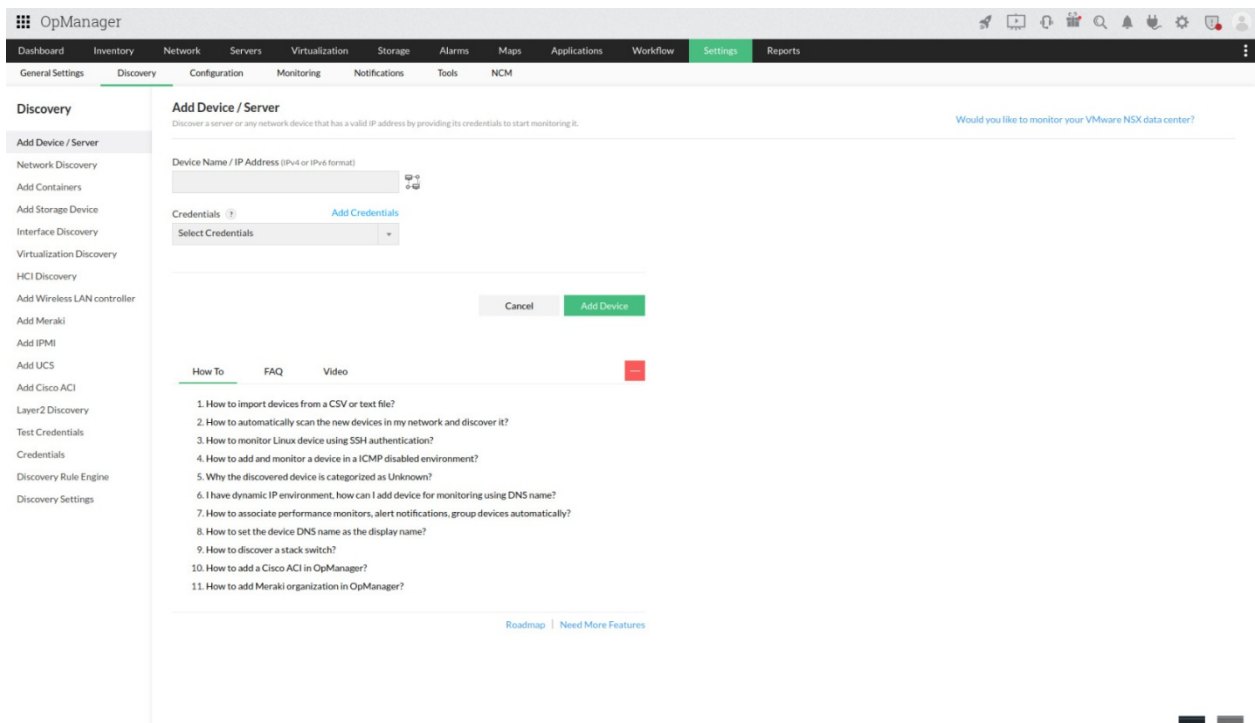


**Figure 4.1 Active Alarms Display Page**

Figure 4.1 depicts the page where the alarm show off for both admin and customer

The alert management module in OpManager provides a centralized view of 'Active Alarms,' highlighting critical issues such as 'File Integrity - Checksum mismatch' and 'File Integrity - New files were found.' These alerts, color-coded by severity, enable administrators to quickly identify and address potential security vulnerabilities and system anomalies within the IT infrastructure.

## 4.2.2 Dashboard & API Integration Module



**Figure 4.2 Dashboard & API Integration Module**

Figure 4.2 depicts the Dashboard & API Integration Module, which visualizes processed device metrics and alerts through an interactive user interface.

It displays key operational parameters in an organized interface to support monitoring and decision-making. The module integrates processed data from backend services and presents it through charts, tables, and status indicators. This dashboard enables users to view system health and take timely actions based on alerts or analytical insights.

## 4.2.3 OID Discovery & Recommendation Module

The screenshot displays the OpManager web interface for configuring device templates. The top navigation bar includes links to Dashboard, Inventory, Network, Servers, Virtualization, Storage, Alarms, Maps, Applications, Workflow, Settings (highlighted), and Reports. Below this, a secondary navigation bar shows General Settings, Discovery, Configuration (highlighted), Monitoring, Notifications, Tools, and NCM. The left sidebar under 'Configuration' lists options: Groups, Device Template (selected), Device Categories, Custom Fields, Alarm Correlation Rules, Alarm Escalation Rules, Interface Templates, Device Downtime Schedules, Device Down Settings, and Quick Configuration Wizard. The main content area is titled 'Device Templates' and features a search bar, 'Add Template', 'Associate', and 'Import' buttons. A 'Show' dropdown is set to 'All'. The table below lists various device templates with their categories and the number of devices associated with each.

| Name                             | Category | Devices | Actions                                  |
|----------------------------------|----------|---------|------------------------------------------|
| 3Com 2000 Access Point           | Wireless | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3com 3226                        | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3com 3500G-EI                    | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com 4200 SuperStack-3           | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com 4200G Switch                | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com 5500                        | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com 6000 Access Point           | Wireless | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com 8000 Access Point           | Wireless | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com a3C5 12                     | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com a3C5 12-withAUI-expn-card   | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com a3C5 12-withBNC-expn-card   | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com a3C5 12-withFOIRL-expn-card | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com accelerators                | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3COM Access Builder              | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3com Access point 8760           | Wireless | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com aDSL Router NoVoDSL Ports   | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com aDSL Router VoDSL Ports     | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3COM AirConnect Access Point     | Wireless | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com application Modules         | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com asterisk Appliance          | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com atm                         | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |
| 3Com ATM Expansion Module        | Switch   | 0       | <a href="#">Add</a> <a href="#">Edit</a> |

**Figure 4.3 Device Templates Configuration Interface**

Figure 4.3 depicts the dashboard section for listing and managing SNMP-based device templates.

This interface enables administrators to view, add, edit, and associate device templates used for monitoring. Once a device is classified based on its sysObjectID, an appropriate template is selected to standardize OID performance metrics collection and alert configuration.

## **4.3 TESTING**

Software testing is the process of evaluating a system to verify that it meets specified requirements and functions reliably under all intended conditions. The primary objective of testing is to identify defects and ensure overall system quality before deployment. For validating the functionality, integration, and performance of the Real-Time SNMP-Based Device Profiling System, multiple testing strategies were applied.

### **4.3.1 Unit Testing**

Unit Testing focuses on testing individual components or functions of the system in isolation to ensure that each core logic unit performs accurately and efficiently. It helps identify issues early in the development process, improving software reliability and reducing the cost of defect correction.

#### **4.3.1.1 Device Discovery Module**

The purpose of unit testing for the Device Discovery Module is to verify that device scanning, IP range validation, and online device detection are functioning accurately. Unit tests ensure correct handling of unreachable devices, timeout operations, and protocol validation before devices are passed to the profiling and polling units.

#### **4.3.1.2 SNMP Polling & OID Processing Module**

Unit testing for the SNMP Polling module validates individual polling operations, OID request execution, response parsing, and trap handling logic. Tests verify correct use of SNMP versions, authentication, retry mechanisms, and error handling to ensure accurate metric retrieval.

#### **4.3.1.3 Device Profiling & Classification Module**

Unit tests ensure that sysObjectID extraction, vendor identification, and device profiling logic function correctly based on predefined rule sets and MIB structure. Individual methods for category mapping and attribute generation are tested for correctness.

#### **4.3.1.4 Alerts & Reporting Module**

Unit testing validates the correctness of threshold rule evaluation, alert trigger conditions, and message formatting. Tests ensure correct logic execution for generating alerts based on CPU, memory, bandwidth, or availability changes.

#### **4.3.1.5 Dashboard API and Database Module**

Unit tests verify insertion and retrieval operations, database schema compliance, and secure REST API communication. CRUD operations, request validation, and data formatting functions are tested individually to ensure reliable storage and display of SNMP metrics.

### **4.3.2 System Testing**

System Testing evaluates the complete, integrated application to ensure it meets functional and performance expectations in realtime execution conditions.

#### **4.3.2.1 Device Discovery Module**

System testing verifies that network scanning, discovery workflow, IP range traversal, and device registration operate correctly and interact seamlessly with the database and dashboard.

#### **4.3.2.2 SNMP Polling & OID Module**

Tests validate real-time polling cycles, trap reception, and metric storage across large-scale device sets, ensuring smooth systemwide polling without data loss or delays.

#### **4.3.2.3 Device Profiling & Classification Module**

System tests ensure correct generation of device profiles, rule engine execution, mapping accuracy, category allocation, and profile updates under various discovery conditions.

#### **4.3.2.4 Alerts & Reporting Module**

Testing verifies real-time alert generation, event logging, scheduled reporting, and delivery through notifications, ensuring accuracy and timely availability of status insights.

#### **4.3.2.5 Dashboard & API Module**

System testing ensures that dashboards accurately reflect live network performance, status indicators, graphs, and history logs while supporting secure multi-user access.

### **4.3.3 Integration Testing**

Integration Testing focuses on verifying the correctness of communication between modules once unit testing has been completed.

#### **4.3.3.1 Device Discovery + Profiling Module**

Integration tests verify smooth data flow from discovery output to device classification, ensuring that discovered devices are properly profiled and updated without conflicts.

#### **4.3.3.2 SNMP Polling + Database + Dashboard API**

Tests ensure that polled device metrics are stored correctly in the database and reflect immediately on the dashboards without inconsistency or latency.

#### **4.3.3.3 Alerts & Reporting + Dashboard**

Integration testing validates that generated alerts are displayed correctly in UI panels, sent through configured channels, and logged accurately for report generation.

#### **4.3.4 Performance Testing**

Performance Testing evaluates system scalability by simulating multiple SNMP devices and polling intervals. Metrics such as response time, resource utilization, network load, and database throughput are measured to ensure system efficiency even under high polling frequency and large device counts.

#### **4.3.5 User Acceptance Testing (UAT)**

UAT verifies that Admin and Customer dashboards support real-time monitoring needs and usability expectations. Feedback is collected from test users to refine interface design, navigation, and alert visibility for improved user experience.

### **4.4 TEST CASES**

A test case is a detailed, step-by-step set of instructions that guides testers in verifying whether a specific feature or functionality of a software application is working as expected. Test cases define the input, execution conditions, expected results, and actual results for a test scenario. [9]

Below are few major reasons to write test cases:

- Ensure correct behavior of each module under different conditions.
- Identify defects early in development, reducing later fixing costs.
- Serve as documentation, detailing how each part works.
- Encourage cleaner, modular, and maintainable code.
- Facilitate regression testing to prevent new bugs.
- Ensure reliable system performance in production, boosting customer confidence.



#### **4.4.1 Challenges Faced During Implementation**

During development, one of the major challenges was handling variations in SNMP support across different device vendors and models, as many devices exposed inconsistent or incomplete OID structures. Ensuring reliable polling under unstable network conditions and minimizing response time delays required extensive retries, timeout optimization, and error-handling mechanisms. Integrating continuous device discovery with real-time metric updates without performance degradation was also challenging, especially when scaling to large device counts. Additionally, mapping sysObjectIDs to accurate profiles demanded collecting and validating vendor MIBs. Designing a dashboard with live updates and role-based access control required efficient backend API management and secure data handling.

**Table 4.1**

| Module                                 | Test Case Summary                                      | Expected Result                                            | Actual Result                                      | Status (Pass/Fail) |
|----------------------------------------|--------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------|--------------------|
| SNMP Communication Module              | Verify SNMP GET request to a device                    | Device responds with valid SNMP data                       | Response returned successfully with correct values | Pass               |
| SNMP Communication Module              | Test timeout handling when device is unreachable       | System should retry and return timeout error without crash | Timeout error handled correctly                    | Pass               |
| sysObjectID Retrieval & Parsing Module | Retrieve sysObjectID from a discovered device          | Correct sysObjectID OID (1.3.6.1.2.1.1.2.0) value returned | Value retrieved successfully                       | Pass               |
| Device Classification Engine           | Classify device based on sysObjectID                   | Correct device type mapped and displayed                   | Device identified as expected                      | Pass               |
| OID Discovery & Recommendation Module  | Perform SNMP walk and list available OIDs              | All device-supported OIDs displayed without failure        | OIDs retrieved and listed                          | Pass               |
| Metrics Collection & Storage Module    | CPU utilization and store in DB                        | Data stored correctly in PERFORMANCE table                 | Data stored and visible in dashboard               | Pass               |
| Alert Management Module                | Trigger alert when threshold exceeds limit (CPU > 90%) | Alert generated and notification visible in dashboard      | Alert triggered successfully                       | Pass               |
| Dashboard & API Integration Module     | Display collected metrics on dashboard                 | Real-time metrics update without delay                     | UI updated correctly                               | Pass               |
| Logging & Audit Module                 | Record polling cycles and system activities            | Logs generated and stored without data gap                 | Logs recorded successfully                         | Pass               |

## **CHAPTER 5**

### **CONCLUSION**

This project successfully implemented a Real-Time SNMP-Based Device Profiling and Monitoring System capable of identifying, classifying, and analyzing network devices dynamically. By integrating modules such as SNMP Communication, sysObjectID Retrieval, Device Classification Engine, OID Discovery, Metrics Collection, Alert Handling, and Dashboard Visualization, the system ensures automated network insight with minimal manual intervention.

The system efficiently retrieves device information through SNMP protocols, classifies them based on sysObjectID, and recommends relevant performance OIDs to enable intelligent monitoring. Real-time data collection and alert mechanisms enhance proactive troubleshooting and reduce network downtime. The dashboard interface enables users to visualize device metrics, alerts, and availability trends quickly and effectively.

Overall, this project demonstrates a scalable and practical solution for modern network environments, reducing dependency on static MIB catalogs and improving operational visibility for administrators. It provides a strong foundation for advanced NMS automation and future integration with larger monitoring platforms.

## **5.1 LIMITATIONS**

### **5.1.1 Limited Vendor Coverage**

Although the system automates device classification using sysObjectID, it may face difficulties with completely new or uncommon vendor devices that do not match known OID patterns. Additional vendor MIB support must be added continuously.

### **5.1.2 Performance Constraints Under Heavy Load**

Polling many devices simultaneously or at very short intervals may impact system performance, increasing response time and network overhead. Optimization and distributed polling mechanisms can improve scalability.

### **5.1.3 Lack of Real-Time Auto-Learning**

The current system does not automatically update classification rules or OID mappings based on live behavior. Machine-learning-based adaptive learning could enhance prediction and recommendation accuracy.

### **5.1.4 Limited Alerts Customization**

Alert thresholds and notification settings are predefined, offering limited flexibility for custom policies based on unique network requirements. A user-configurable rules engine would improve operational control.

### **5.1.5 Dashboard Dependency on Network Connectivity**

Since the dashboard updates rely on real-time data transmission, any network latency or interruption may temporarily impact visualization accuracy or delay updates.

## **5.2 FUTURE ENHANCEMENTS**

### **AI-Based Device Classification**

Implement machine learning models to analyze device metadata, OID patterns, and performance signatures to automatically classify unknown devices with higher accuracy. This reduces dependency on static sysObjectID mappings and enables faster onboarding of new hardware.

### **Automated MIB Fetching & Template Generation**

Integrate a mechanism to automatically download vendor MIB files, parse their OID structures, and generate monitoring templates dynamically. This expands device support and ensures comprehensive metric coverage without manual updates.

### **Distributed Polling & Scalability Improvements**

Enhance the architecture with multiple distributed pollers to balance SNMP load, improve performance for large networks, and ensure high availability. This makes the system more scalable and reliable for enterprise-level deployments.

## REFERENCES

**SNMP, SNMPv2, and RMON: Practical Network Management** — William Stallings.

A comprehensive guide covering SNMP versions and RMON, ideal for understanding network monitoring fundamentals and standard MIB/OID usage.

**Essential SNMP** — Douglas Mauro & Kevin Schmidt. A practical manual for administrators implementing SNMP: covers agents, managers, traps, MIBs, community strings, and real-world deployment tips.

**SNMP Network Management** — Paul Simoneau. Provides hands-on guidance for using SNMP to manage TCP/IP networks, useful when building or customizing SNMP-based management solutions.

**SNMP: A Guide to Network Management** — Sidnie M. Feit. Focused on practical SNMP-based network administration, this book explains what can be managed via SNMP and how to interpret management data — good for understanding device metadata retrieval and interpretation.

**Network Management Fundamentals** — by the Cisco Press Fundamentals Series. Offers a solid foundation in network management principles, covering topologies, protocols, and monitoring techniques beyond just SNMP — suited for designing scalable monitoring frameworks.